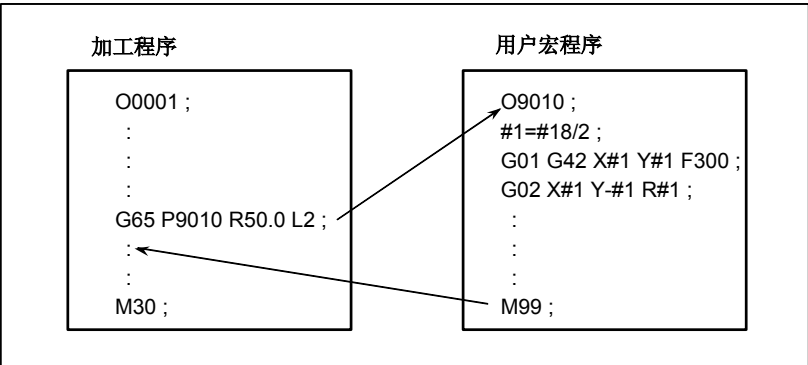


16 用户宏程序

虽然子程序对一个重复操作很有用，但若使用用户宏程序功能，则还可以使用变量、运算指令以及条件转移，使一般程序（如压坑加工和用户自定义的固定循环等）的编写变得更加容易。

加工程序可以用一个简单的指令调用用户宏程序，就像调用子程序一样。



16.1 变量

普通的加工程序直接用数值指定 G 代码和移动量，例如 G00、X100.0。
使用用户宏程序时，除了可直接指定数值外，还可以指定变量号，可通过程序或 MDI 面板上操作来改变该数值。

```
#1=#2+100 ;  
G01 X#1 F300 ;
```

解释

• 变量的表达方式

当指定一个变量时，在符号“#”的后面指定变量号。

#i (i=1,2,3,4,.....)

〔例〕 #5

#109

#1005

或者使用后面将要叙述的“运算指令”项目中的<表达式>，按照如下方式表达。

#[<表达式>]

〔例〕 #[#100]

#[#1001-1]

#[#6/2]

下面说明中的变量#i，可以用变量 #[<表达式>]来替换。

• 变量的种类

根据变量号，可以将变量分为局部变量、公共变量、系统变量，各类变量的用途和特性各不相同。另外，我们还为用户准备了只读专用的系统变量。

• 变量的范围

局部变量和公共变量可使用下列范围内的任意值。如果运算结果超过此范围，就会有报警（PS0111）发出。

参数 F16(No.6008#0)=0 时

最大值：约±10³⁰⁸

最小值：约±10⁻³⁰⁸

用户宏程序中进行处理的数值数据，基于 IEEE 标准，作为双倍精度实数处理。运算过程中出现的误差，也基于此双倍精度。

参数 F16(No.6008#0)=1 时

最大值：约±10⁴⁷

最小值：约±10⁻²⁹

- 局部变量(#1~#33)

局部变量就是在宏内被局部使用的变量。也即，它与在某一时刻调用的宏中使用的局部变量 #i 和在另一时刻调用的宏（不管是以前的宏，还是别的宏）中使用的 #i 不同。因此，在如多层调用一样从宏 A 中调用宏 B 时，有可能在宏 B 中错误使用在宏 A 中正在使用的局部变量，导致破坏该值。

局部变量用于传输自变量。其与自变量地址之间的对应关系，请参阅宏程序调用指令的章节。没有被传输自变量的局部变量，在初始状态下为<空值>，用户可以自由使用。局部变量的属性为可 READ/WRTIE（读/写）。

- 公共变量(#100~#199、#500~#999)

局部变量在宏内部被局部使用，而公共变量，则是在主程序、从主程序调用的各子程序、各个宏之间通用。也即，在某一宏中使用的 #i 与在其他宏中使用的 #i 是相同的。此外，由某一宏运算出来的公共变量#i，可以在别的宏中使用。局部变量的属性，基本上为可 READ/WRTIE（读/写）。但是，也可以对由参数 (No.6031~6032)指定的变量号的公共变量进行保护（设定为只读）。公共变量的用途没有在系统中确定，因此，用户可以自由使用。公共变量的个数，可通过选项，从下列 2 类中选择。

- (a)100 个公共变量（在用户宏程序选项中使用）

可以使用公共变量#100~#149 和#500~#549。#100~#149 将会由于电源切断而被清除，但是，#500~#549 即使在电源切断之后仍会被保留起来。

- (b)600 个公共变量（可通过用户宏程序以及用户宏程序公共变量追加的选项使用）

可以使用公共变量#100~#199 和#500~#999。

#100~#199 将会由于电源切断而被清除，但是，#500~#999 即使在电源切断之后仍会被保留起来。

- 公共变量的写保护

通过在参数(No.6031~6032)中设定变量号，即可对多个公共变量(#500~#999)进行保护，也即将其属性设定为只读。此功能对利用 MDI 从宏画面的输入 / 全部清零、在宏程序中的写入均有效。利用 NC 程序将设定范围的公共变量设定为 WRITE（在左边使用）时，会有报警(PS0116)发出。

- 系统变量

系统变量是在系统中其用途被固定的变量。其属性共有 3 类：只读、只写、可读/写，根据各系统变量而属性不同。

- 系统常量

我们为用户准备了其值不变的常量，用户可以与变量一样地引用这些常量。系统常量的属性为只读。

- 小数点的省略

在程序中定义变量值时，可省略小数点。

〔例〕#1=123；的含义是：变量#1 的实际值是 123.000。

• 变量的引用

可以用变量指定紧接地址之后的数值。如果编制一个<地址>#i 或<地址>-#i 的程序，则意味着原样使用变量值，或者将其补码作为该地址的指令值。

- （例）当 F#33、#33=1.5 时，与指定了 F1.5 时的情形相同。
- 当 Z-#18、#18=20.0 时，与指定了 Z-20.0 时的情形相同。
- 当 G#130、#130=3.0 时，与指定了 G3 时的情形相同。

不可引用地址/、:、O 和 N 中的变量。

- （例）不可编制诸如 O#27 或 N#1 的程序。
- 不可将可选程序段跳转/n 的 n（n=1~9）作为变量来使用。

不能直接用变量来指定变量号。

- （例）用#30 来替换#5 的 5 时，代之以指定##30，指定#[#30]。
- 不能指定超过每个地址中所确定的最大指令值的值。

- （例）当#140=120 时，G#140 超过最大指令值。
- 变量为地址数据时，变量被自动地四舍五入到各地址有效位数以下的位数。

- （例）在设定单位为 1/1000mm(IS-B)的装置上，#1 为 12.3456 时，如果执行 G00 X#1；，实际指令将成为 G00 X12.346；

利用后面叙述的<表达式>，可以用<表达式>来替换紧跟在地址之后的数值。

<地址>[<表达式>] 或<地址>- [<表达式>]

若按照上面的顺序编程，则意味着原样使用<表达式>的值，或者将其补码作为该地址的指令值。需要注意的是，[] 中使用的不带小数点的常量，视为其末尾带有小数点。

- （例）X[#24+#18 * COS[#1]]
- Z-[#18+#26]

• 未定义变量

我们将尚未定义变量值的状态叫做“空值”。
变量#0、#3100 永远是空变量，它不能写入，但能读取。

(a) 引用变量

在引用一个尚未定义的变量时，地址本身也被忽略。

原来的指令	G90 X100 Y#1
#1=<空值>时的等效指令	G90 X100
#1=0 时的等效指令	G90 X100 Y0

(b) 定义 / 替换、加法运算、乘法运算、

将局部变量或公共变量直接替换为<空值>时，其结果也为<空值>。将系统变量直接替换为<空值>时，或者替换使用<空值>运算出来的结果是，均作为变量值 0 来对待。

原来的运算式子（局部变量例）	#2=#1	#2=#1*5	#2=#1+#1
替换结果（#1=<空值>时）	<空值>	0	0
替换结果（#1=0 时）	0	0	0

原来的运算式子 (公共变量例)	#100=#1	#100=#1*5	#100=#1+#1
替换结果 (#1=<空值>时)	<空值>	0	0
替换结果 (#1=0 时)	0	0	0

原来的运算式子 (系统变量例)	#2001=#1	#2001=#1*5	#2001=#1+#1
替换结果 (#1=<空值>时)	0	0	0
替换结果 (#1=0 时)	0	0	0

(c) 比较运算

若是 EQ 和 NE 的情形，<空值>和 0 被判定为不同的值。

若是 GE、GT、LE、LT 的情形，<空值>和 0 被判定为相同的值。

- 将<空值>代入#1 时

条件表达式的 表达方式	#1 EQ #0	#1 NE 0	#1 GE #0	#1 GT 0	#1 LE #0	#1 LT 0
评价结果	成立 (真)	成立 (真)	成立 (真)	不成立 (假)	成立 (真)	不成立 (假)

- 将 0 代入#1 时

条件表达式的 表达方式	#1 EQ #0	#1 NE 0	#1 GE #0	#1 GT 0	#1 LE #0	#1 LT 0
评价结果	不成立 (假)	不成立 (假)	成立 (真)	不成立 (假)	成立 (真)	不成立 (假)

- 系统变量（常量）的名称指令

系统变量（常量）通过变量号指定，但是，也可以通过事先预备的系统变量（常量）名称来指定。系统变量（常量）名称由_（下划线）开始的8个字符以内的英文大写字母、数字以及_（下划线）构成。另外，依赖于轴的变量（坐标值等）或存在多个同类数据的变量（刀具补偿值等），作为名称的下标，可以用[n]（n为整数）指定数值。此时，n可以用<表达式>即运算格式指定。指令格式为

[# DATE]

务须以 [# 系统变量名称] 的格式指定。

〔例〕`[# DATE]=20040117;` : 將 2004.01.17 代入#3011 (年月日)

[# TIME]=161705 ; : 将 16:17:05 代入#3012 (时分秒)

#101=[#_ABSMT[1]]； ： 读出#5021（第1轴的机床坐标值），代入
#101

#102=[#_ABSKP[#500*2]] ; 读出#506x (第[#500*2]轴的跳转位置),
代入#102

为下标 **n** 指定了整数值以外的值时, 视为指定了将 **n** 的小数点以下进行四舍五入处理后的值并引用变量值。

(例) [# ABSIO[1.4999999]]: 假定此值为[# ABSIO[1]], 也即#5001。

[# ABSIO[1.5000000]]: 假定此值为[# ABSIO[2]], 也即#5002。

注释

- 1 指定尚未记录的变量名称时，会有报警(PS1098)发出。
- 2 指定了非法的值作为下标 n（负数等）时，会有报警(PS1099)发出。

16.3 运算指令

可以在变量之间进行各类运算。运算指令可像一般的算术式子一样编程。

#i=<表达式>

<表达式>

运算指令右边的<表达式>是常量、变量、函数或算符的组合。代之以下面的#j、#k，也可以使用常量。在<表达式>中使用的不带数点的常量，视为其末尾有小数点。

表16.3 (a) 运算指令

运算的种类	运算指令	含义
①定义、替换	#i=#j	变量的定义或替换
②加法型运算	#i=#j+#k #i=#j-#k #i=#j OR #k #i=#j XOR #k	加法运算 减法运算 逻辑和（32 位的每一位） 按位加（32 位的每一位）
③乘法型运算	#i=#j*#k #i=#j/#k #i=#j AND #k #i=#j MOD #k	乘法运算 除法运算 逻辑积（32 位的每一位） 余数（#j、#k 取整后求取余数。#j 为负时，#i 也为负。）
④函数	#i=SIN[#j] #i=COS[#j] #i=TAN[#j] #i=ASIN[#j] #i=ACOS[#j] #i=ATAN[#j] #i=ATAN[#j]/[#k] #i=ATAN[#j,#k] #i=SQRT[#j] #i=ABS[#j] #i=BIN[#j] #i=BCD[#j] #i=ROUND[#j] #i=FIX[#j] #i=FUP[#j] #i=LN[#j] #i=EXP[#j] #i=POW[#j,#k] #i=ADP[#j]	正弦（deg 单位） 余弦（deg 单位） 正切（deg 单位） 反正弦 反余弦 也可以是反正切（1 个自变量）、ATN。 也可以是反正切（2 个自变量）、ATN。 同上 也可以是平方根、SQR。 绝对值 由 BCD 变换为 BINARY 由 BINARY 变换为 BCD 也可以是四舍五入、RND。 小数点以下舍去 小数点以下舍入 自然对数 以 e（2.718……）为底数的指数 幂乘级（#j 的 #k 乘级） 小数点附加

解释

• 角度单位

在 SIN、COS、TAN、ASIN、ACOS、ATAN 函数中使用的角度，其单位用度表示。

比如，90 度 30 分，表示为 90.5 度。

• 反正弦 $\#i = \text{ASIN}[\#j]$ ；

- 解的范围如下所示。
当参数 NAT (No.6004#0) 设为 0 时： $270^\circ \sim 90^\circ$
当参数 NAT (No.6004#0) 设为 1 时： $-90^\circ \sim 90^\circ$
- 当 $\#j$ 超出-1~1 时，会有报警 (PS0119) 发出。
- 可用常量替代变量 $\#j$ 。

• 逆余弦 $\#i = \text{ACOS}[\#j]$ ；

- 解的范围为 $180^\circ \sim 0^\circ$ 度。
- 当 $\#j$ 超出-1~1 时，会有报警 (PS0119) 发出。
- 可用常量替代变量 $\#j$ 。

• 反正切 $\#i = \text{ATAN}[\#j]/[\#k]$ ；(2 个自变量)

- 即使指定 $\text{ATAN}[\#j, \#k]$ ，也与 $\text{ATAN}[\#j]/[\#k]$ 等效。
- 本函数由 X-Y 平面上的点($\#k, \#j$)给定时，返还相对于由该点构成的角度的反正切的值。
- 可用常量替代变量 $\#j$ 。
- 解的范围如下所示。
当参数 NAT (No.6004#0) 设为 0 时： $0^\circ \sim 360^\circ$
[例] $\#1 = \text{ATAN}[-1]/[-1]$ ；时， $\#1$ 是 225.0
当参数 NAT (No.6004#0) 设为 1 时： $-180^\circ \sim 180^\circ$
[例] $\#1 = \text{ATAN}[-1]/[-1]$ ；时， $\#1$ 是-135.0

• 反正切 $\#i = \text{ATAN}[\#j]$ ；(1 个自变量)

- 如上所述，在以 1 个自变量来指定 ATAN 时，该功能将返还反正切的主值 ($-90^\circ \leq \text{ATAN}[\#j] \leq 90^\circ$)。也即，成为计算器型规格的 ATAN。
- 在将本函数作为除法运算的被除数使用时，务须以 [] 括起来以后再指定。不包括起来的情形视为 $\text{ATAN}[\#j]/[\#k]$ 。
(例) $\#100 = [\text{ATAN}[1]]/10$ ；：将 1 个自变量 ATAN 除以 10。
 $\#100 = \text{ATAN}[1]/[10]$ ；：作为 2 个自变量 ATAN 执行。
 $\#100 = \text{ATAN}[1]/10$ ；：视为 2 个自变量 ATAN，但是由于 X 坐标的指定中没有 []，会有报警(PS1131)发出。

• 自然对数 $\#i = \text{LN}[\#j]$ ；

- 当逆对数 ($\#j$) 小于等于 0 时，会有报警 (PS0119) 发出。
- 可用常量替代变量 $\#j$ 。

- 指数函数#i=EXP[#j];

- 运算结果溢出时，会有报警(PS0119)发出。
- 可用常量替代变量#j。

- ROUND 函数

- 当运算指令以及 IF 语句或 WHILE 语句中包含取整函数时，取整函数则从第一位小数起四舍五入。

[例] #1=ROUND[#2]; 其中#2 为 1.2345，则变量#1 的值是 1.0。

- 当 ROUND 函数用在 NC 的语句地址中，取整函数将按地址的最小输入增量，对指定的值四舍五入。

[例] 按照变量#1 和#2 的值，编写一个钻孔程序，完成后返回原来位置。
假设设定单位是 1/1000mm，变量#1 的值是 1.2345，变量#2 的值是 2.3456。

G00 G91 X-#1; 负向移动 1.235mm。

G01 X-#2 F300; 负向移动 2.346mm。

G00 X[#1+#2]; 由于 1.2345+2.3456=3.5801，移动量在正向为 3.580mm，而后返回原来位置。

差别来源于：是四舍五入前相加，还是四舍五入后相加。

为了使刀具返回原点，必须指定 G00 X[ROUND[#1]+ ROUND[#2]];

- 关于 ADP (Add Decimal Point) 函数

- 设定 ADP[#n](n=1~33)，对于不带小数点传递的自变量，可以在子程序端添加小数点。

(例) 在用 G65 P_X10；调用的子程序端，ADP[#24]的值与在自变量的最后带有小数点的情形相同，也即为 10.。它在不想在子程序端考虑设定单位时使用。但是，参数 CVA (No.6007#4) 为 1 时，自变量在传递的同时被变换为 0.01，因此不可使用 ADP 函数。

注释

出于程序的兼容性考虑，建议用户不使用 ADP 函数，在指定宏程序调用的自变量中添加上小数点。

- 只入不舍和只舍不入 (FUP 和 FIX)

当 CNC 对一个数进行操作后，其整数的绝对值比该数原来的绝对值大，这种操作称只入不舍；相反，对一个数进行操作后，其整数的绝对值比该数原来的绝对值小，这种操作称为只舍不入。

当处理负数时，要格外小心。

[例] 假设 #1=1.2，#2=-1.2

当执行 #3=FUP[#1]时，将 2.0 赋予#3。

当执行 #3=FIX[#1]时，将 1.0 赋予#3。

当执行 #3=FUP[#2]时，将-2.0 赋予#3。

当执行 #3=FIX[#2]时，将-1.0 赋予#3。

• 运算指令的缩写

当函数在程序中被指定时，只需要前面的两个字符，后面的可以省略。

[例] ROUND→RO
FIX → FI 等

注释
POW 不可省略。

• 运算的优先顺序

- ①函数
- ②乘除运算 (*, /, AND)
- ③加减运算 (+, -, OR, XOR)

例) #1=#2+#3*SIN[#4];

①②③表示运算的顺序。

• 括号的嵌套

括号被用来改变运算的优先顺序。
括号可含 5 层，包括函数外面的括号。
因为超过 5 位，会有报警(PS0118)发出。

例) #1=SIN [[[#2+#3] * #4+#5] * #6] ;

①~⑤表示运算的顺序。

限制

- 有关精度下降的注意事项
参数 F16(No.6008#0)=0 时

- 加减法运算

在加法或减法运算中，需要注意的是，当绝对值被减掉时，不能将相对误差控制在低于 10^{-15} 。

比如，假定#1 和#2 的真值在运算过程中成为如下所示的情形。

（这是运算过程中的例子，不可从实际程序进行下述指令）

#1=9876543210.987654321

#2=9876543210.987657777

进行运算#2-#1

#2-#1=0.000003456

不可得出上面的结果。原因在于，用户宏程序变量只有 10 进制 15 位数的精度，#1、#2 的值分别具有如下精度：

#1=9876543210.987650000

#2=9876543210.987660000

（由于其内部为二进制，其精度与上述值有所不同。）因此，成为如下结果：

#2-1=0.000010000

并由此产生较大的误差。

- 逻辑关系

EQ、NE、GT、LT、GE、LE 基本上与加/减法运算相通，需要注意其误差。比如，在上例中，要判断#1 和#2 是否相等，如果用

IF [#1 EQ #2]

就会因为误差的关系而不得能够作出正确的判断。

IF [ABS [#1-#2]LT 0.1]

如上所示，必须评价此时的误差，如果#1 和#2 的差在该误差的范围内，则可以认为其值相等。

- 三角函数

三角函数中，虽然其绝对误差得到保证，但是，由于相对误差不到 10^{-15} ，在三角函数运算后进行乘法运算和除法运算时需要引起注意。

- FIX 函数

对于运算结果使用 FIX 函数是，也同样需要注意其精度。比如，假定下式成立，

N10 #1=0.002 ;

N20 #2=#1*1000 ;

N30 #3=FIX[#2] ;

#3 不见得就成为 2。

这是因为在 N20 的运算中产生误差，导致

#2 不会等于 2.0000000000000000，

如#2=1.999999999999997 所示，

该值有可能小于 0。为了预防这种情况，应按照如下所示设定 N30。

N30 #3=FIX[#2+0.001];

通常，

FIX[表达式] → FIX[表达式±ε]

（表达式的值为正时指定+ ε ，为负时指定- ε ， ε 应根据需要设定诸如0.1、0.01、0.001、……等）这样的值。

注释
当#j 大约超过 790 时，指数函数#i=EXP[#j]；的运算结果溢出。

参数 F16(No.6008#0)=1 时

运算时可能会产生误差。

表16.3（b） 相对于各运算形式的运算误差

运算形式	平均误差	最大误差	误差的种类
a = b * c	1.55×10^{-10}	4.66×10^{-10}	相对误差 (注释 1) $\left \frac{\varepsilon}{a} \right $
a = b / c	4.66×10^{-10}	1.88×10^{-9}	
a = $\sqrt{b}$	1.24×10^{-9}	3.73×10^{-9}	
a = b + c a = b - c	2.33×10^{-10}	5.32×10^{-10}	(注释 2) $\text{MIN} \left \frac{\varepsilon}{b} \right , \left \frac{\varepsilon}{c} \right $
a = SIN [b] a = COS [b]	5.0×10^{-9}	1.0×10^{-8}	绝对误差 (注释 3) $\left \varepsilon \right $ 度
a = ATAN [b] / [c]	1.8×10^{-6}	3.6×10^{-6}	

注释
1 相对误差与运算结果有关。
2 两类误差中，采用较小的一类。
3 绝对误差是常量，与运算结果无关。
4 函数 TAN 运算的是 SIN/COS。
5 自然对数#i=LN[#j]；、指数函数#i=EXP[#j]；由于无法将相对误差控制在低于 10-8 的值，应予以注意。
6 当#j 大约超过 110 时，指数函数#i=EXP[#j]；的运算结果溢出。

- 变量值的精度大约是 8 位十进制数字，当对非常大的数进行加法或减法运算时，可能得不到预期的结果

〔例〕 #1=9876543210123.456

即使#2=987654327777.777 为真值，

变量值为#1=9876543200000.000

#2=9876543300000.000。

此时，在计算#3=#2-#1；时，#3=100000.000 。

(实际上由于是二进制而存在一定的差异。)

- 注意：误差还会来自使用 EQ、NE、GE、GT、LE、LT 的条件表达式。

〔例〕 IF[#1 EQ #2]受#1 和#2 的误差影响，可能导致判断错误。

因此，如 IF[ABS[#1-#2] LT 0.001] 所示，求出两个变量之差，如果该差值不超过容许值（在这个例子中为 0.001），则可以判定这两个变量的值相等。

- 对一个数值进行只舍不入时，应小心。

(例) #1=0.002;

计算#2=#1*1000; 时, #2 的结果不正好是 2, 而是 1.99999997。

此时, 如果指定#3=FIX[#2]; 变量#3 的结果值不正好是 2.0, 而是 1.0。

在这种情况下, 进行误差修正之后, 对该值应进行只舍不入或四舍五入, 使结果大于预期的整数值。

#3=FIX[#2+0.001];

#3=ROUND[#2];

• 括号

在表达式中使用的括号为方括号[]。

注意: 圆括号()用于注释。

• 除数

除法运算中分母为“0”时, 会有报警(PS0112)发出。

16.5 宏语句和 NC 语句

下列程序段为宏语句：

- 含运算指令(=)的程序段
- 含控制指令(如 GOTO、DO、END)的程序段
- 含宏程序指令(如由 G65、G66、G66.1、G67 或别的 G 代码，或 M 代码的宏程序)的程序段

除宏语句以外的程序段称为 NC 语句。

解释

• 与 NC 语句的区别

- 即使在单程序段方式，机床也不停止。
然而，应注意：当参数 SBM (No.6000#5) 设为 1 时，在单程序段方式机床会停止。
- 将其视为刀具半径补偿方式中的没有移动的程序段。

• 与宏语句有相同性质的 NC 语句

- NC 语句为子程序调用指令 (M98，采用 M 代码的子程序调用，或采用 T 码的子程序调用)，且不含 O 或文件名、N、P、L 以外的指令地址的程序段，与宏语句具有相同的性质。
- 包含 M99，且不包含 O 或文件名、N、L、P 以外的指令地址的程序段之 NC 语句，与宏语句具有相同的性质。

16.6 转移和重复

在程序中，可用 GOTO 语句和 IF 语句改变程序的流向。
转移和重复有下列三种。

转移和重复		GOTO	（无条件转移）
		IF 语句	（条件转移、如果…，那么…）
		WHILE	（重复、～之间）

16.6.1 无条件转移（GOTO 语句）

无条件转移到顺序号为 n 的语句。
当顺序号在 1～99999999 范围以外，就会有报警（PS0128）发出。
另外，顺序号也可用表达式来指定。

GOTOn ;	n：顺序号（1～99999999）
----------------	--------------------------

〔例〕GOTO 1;
GOTO #10;

 **警告**
不可在一个程序中指定多个附带有相同顺序号的程序段。利用 GOTO 语句转移时，转移目的地不定，十分危险。

注释
1 反向转移比正向转移需要更长的时间。
2 在 GOTOn 后面将被执行的顺序号 n 的程序段中，顺序号必须在程序段的开头。当顺序号不在程序段的开头时不可转移。

16.6.2 顺序号存储型 GOTO 语句

在执行用户宏程序控制指令 GOTO 语句时，对于以前执行并存储的顺序号，高速地进行顺序号搜索。

“以前执行并存储的顺序号”，即就所执行的顺序号，在相同程序内存储不重复的顺序号以及子程序调用的顺序号。

存储类型因下面的参数而不同。

- (1) 参数 MGO(No.6000#1)=1 时
 - 固定类型… 从开始运行起执行并存储的最多 20 个顺序号。
- (2) 参数 HGO(No.6000#4)=1 时
 - 可变类型… 在执行 GOTO 语句前就执行并存储的最多 30 个顺序号。
 - 履历类型… 以前曾利用 GOTO 语句进行顺序号搜索并存储的最多 10 个顺序号。

存储的顺序号在下列情形下将会被取消。

- 刚刚接通电源后。
- 进行复位时。
- 在进行程序的记录和编辑（含后台编辑以及 MDI 程序的编辑）运行程序时。

 **警告**

请勿在一个程序中指定多个带有相同顺序号的程序段。
如果在 GOTO 语句的前后指定与转移目的地顺序号相同的顺序号并执行 GOTO 语句时，转移目的地会因参数而发生下例所示的变化，此种情形将十分危险。

参数 MGO (No.6000#1) 参数 HGO (No.6000#4) 中的任何一者为 1 时	参数 MGO (No.6000#1) 参数 HGO (No.6000#4) 均为 0 时
<pre> : N10; ← 转移到 GOTO : 语句前的 N10 GOTO10; : N10;</pre>	<pre> : N10; : GOTO10; ← 转移到 GOTO : 语句后的 N10 N10;</pre>

即使参数 MGO(No.6000#1)、参数 HGO(No.6000#4)的其中之一为 1，在执行 GOTO 语句时，如果以前执行并存储的顺序号中没有转移目的地的顺序号，就转移到 GOTO 语句的程序段之后的顺序号（两者都为 0 时相同）。

注释

在通过 DNC 运行由外部一边读入程序一边执行时，不会存储所执行的顺序号。
但是，在执行通过子程序调用等记录在存储器中的程序时，则存储所执行的顺序号。

 **注意**

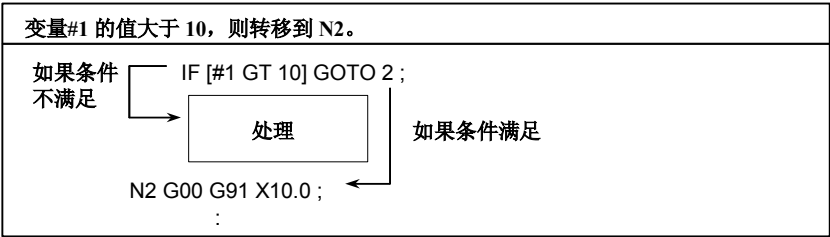
由于 GOTO 语句的限制，不可转移到 DO~END 环路内。如果编制了转移到环路内的程序，则使用顺序号存储型 GOTO 语句的情形和不使用顺序号存储型 GOTO 语句的情形，其操作不同。

16.6.3 条件转移（IF 语句）

在 IF 后指定<条件表达式>。

IF[<条件表达式>]GOTO n

如果指定的<条件表达式>（真）满足，则转移到述顺序号为 n 的语句；如果条件表达式不满足，程序执行下一程序段。



IF[<条件表达式>]THEN

如果<条件表达式>成立（真），则执行指定在 THEN 之后的宏语句。
但只执行一个宏语句。

当#1 和#2 一致时，将 0 代入#3。
IF[#1 EQ #2] THEN#3=0 ;

当#1 和#2 一致，且#3 和#4 一致时，将 0 代入#5。
IF[[#1 EQ #2] AND [#3 EQ #4]] THEN#5=0 ;

当#1 和#2 一致，且#3 和#4 一致时，将 0 代入#5。
IF[[#1 EQ #2] OR [#3 EQ #4]] THEN#5=0 ;

解释

• <条件表达式>

<条件表达式>有两种：<简单条件表达式>和<复合条件表达式>。<简单条件表达式>即在相比较的 2 个变量或变量和常量之间描述表 16.6(a)所示的比较算符的条件表达式。代之以变量，也可以描述<表达式>。<复合条件表达式>即将多个<简单条件表达式>的真假结果以 AND（逻辑积）、OR（逻辑和）、XOR（按位加）进行运算的结果。

• 比较算符

每个算符由两个字母组成，用来比较两个值，决定它们是否相等，或一个值比另一个值小或大。需要注意的是，等号（=）、不等号（>、<）不可作为比较算符使用。

表16.6 (a) 比较算符

算符	含义
EQ	等于（=）
NE	不等于（≠）
GT	大于（>）
GE	大于或等于（≥）
LT	小于（<）
LE	小于或等于（≤）

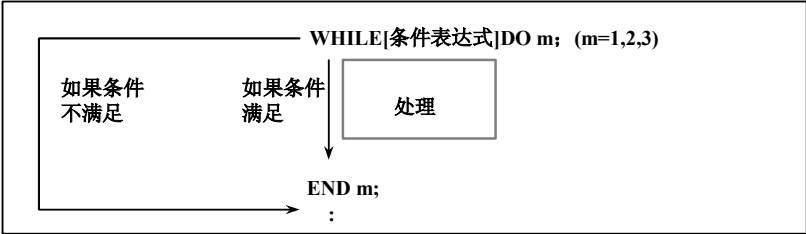
程序例子

下面的程序例求出 1～10 之和。

```
O9500;
  #1=0; ..... 解的初始值
  #2=1; ..... 加数的初始值
N1 IF[#2 GT 10] GOTO 2; ..... 加数超过 10 时就转移到 N2
  #1=#1+#2; ..... 计算解
  #2=#2+1; ..... 下一个加数
  GOTO 1; ..... 转移到 N1
N2 M30; ..... 程序的结尾
```

16.6.4 重复（WHILE 语句）

在 WHILE 后指定条件表达式。
当指定的条件表达式满足时，执行从 DO 到 END 之间的程序。
当指定的条件表达式不满足时，进入 END 后面的程序段。

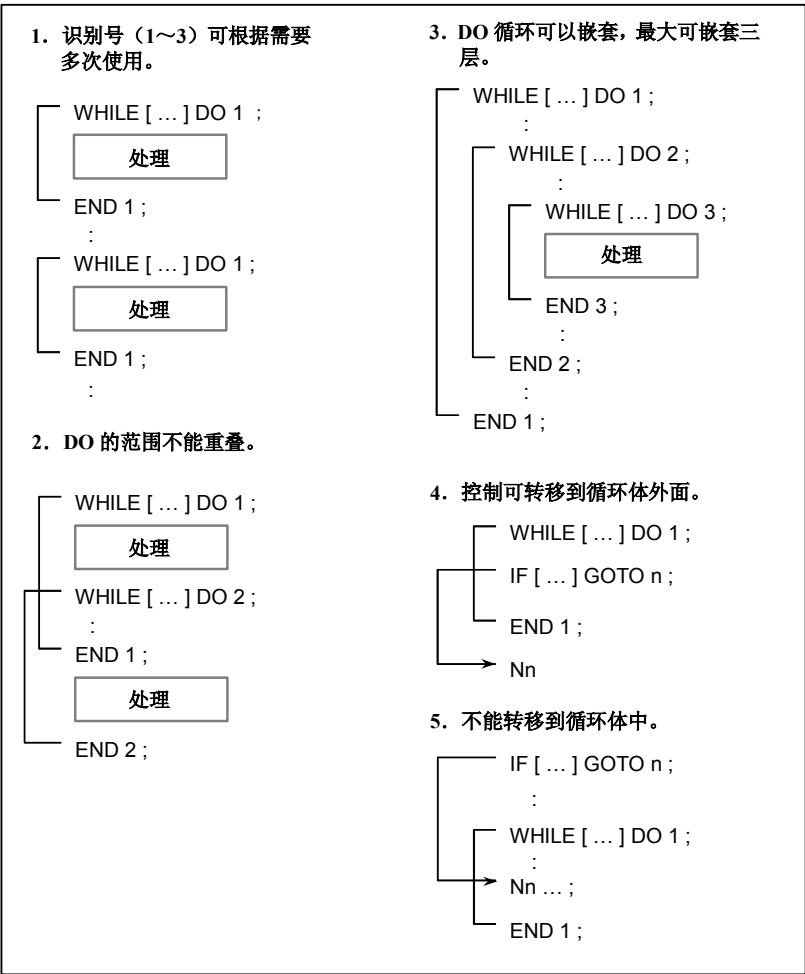


解释

当指定的条件表达式满足时，执行紧跟 WHILE 后的从 DO 到 END 之间的程序。
当指定的条件表达式不满足时，执行与 DO 对应的 END 后面的程序段。
条件表达式和算符与 IF 语句相同。
DO 和 END 后面的数值是指定执行范围的识别号，可用 1、2、3 作为识别号。
如果用 1、2、3 以外的数字作为识别号，则会有报警(PS0126)发出。

• 嵌套

识别号（1~3）在 DO~END 之间可多次使用。
但是，当重复的循环相互交叉时，会有报警（PS0124）发出。



限制**• 无限循环**

当指定 DO m 而省略 WHILE 语句时，程序则在 DO 到 END 之间无限循环。

• 处理时间

当要转移到 GOTO 语句中指定的顺序号时，程序先搜索顺序号。为此，反向处理数据比正向要用较长的时间。因此，反向处理数据时，为了缩短处理时间，作为重复指令，请使用 WHILE 语句。

• 未定义变量

在使用 EQ、NE 的条件表达式中，<空值>和 0（零）有不同的效果，在别的条件表达式中，<空值>被看成 0。

程序例子

下面的程序例求 1 至 10 之和。

```
O0001;  
#1=0;  
#2=1;  
WHILE[#2 LE 10]DO 1;  
#1=#1+#2;  
#2=#2+1;  
END 1;  
M30;
```

16.7 宏程序调用

可用下列方法调用宏程序。调用方法大致可分为 2 类：宏程序调用和子程序调用。即使在 MDI 运行中，也同样可以调用程序。



限制

• 调用的嵌套

调用的嵌套，仅宏程序调用为 5 层，仅子程序调用为 10 层，共计 15 层。

• 宏程序调用与子程序调用的差别

宏程序调用 (G66/G66.1/Ggg/Mmm) 与子程序调用 (M98/Mmm/Ttt 等) 具有如下差别：

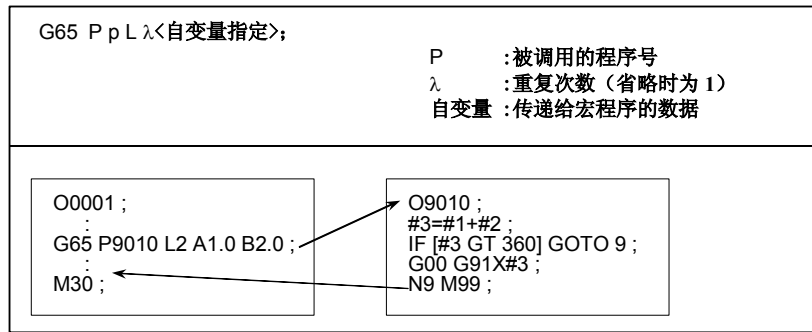
- 宏程序调用可指定自变量（传递给宏程序的数据），子程序调用则不能。
- 宏程序调用的程序段中含有其他的 NC 指令(如 G01 X100.0 G65 Pp)时，会有报警(PS0127)发出。
- 当子程序调用的程序段包含其他的 NC 指令时（如 G01 X100.0 M98 Pp）时，执行该指令后调用子程序。
- 宏程序调用的程序段不会在单程序段方式下停止。
宏程序调用的程序段中含有其他的 NC 指令(如 G01 X100.0 M98 Pp)时，在单程序段方式停止。
- 宏程序调用会引起局部变量的级别变化，但子程序调用不会引起变化。（限制见“局部变量的级别”）

• 被调用的程序和文件夹之间的关系

根据子宏程序调用和程序调用的方法，决定搜索文件夹的顺序。
按照顺序搜索文件夹，调用最先找到的程序。
详情请参阅“程序的管理”章节。

16.7.1 简单调用（G65）

当指定 G65 时，指定在地址 P 处的用户宏程序被调用。
另外，数据（自变量）被传递给用户宏程序。



解释

- 调用
 - 在 G65 后面，由地址 P 指定将要调用的用户宏程序的程序号。
 - 需要指定重复次数时，在地址 L 后，指定重复次数（1～999999999）的范围。如果省略 L，则假设为 1。
 - 用自变量指定法，将其值代入相对应的局部变量中。
- 自变量指定法

有两类自变量指定法：第Ⅰ类使用 G、L、O、N、P 以外的字母，每个用一次；第Ⅱ用 A、B、C，每个用一次，还可使用 10 组 I、J、K，自变量的指定种类是根据所用的字母自动决定的。

第Ⅰ类自变量指定法

地 址	变量号	地 址	变量号	地 址	变量号
A	#1	I	#4	T	#20
B	#2	J	#5	U	#21
C	#3	K	#6	V	#22
D	#7	M	#13	W	#23
E	#8	Q	#17	X	#24
F	#9	R	#18	Y	#25
H	#11	S	#19	Z	#26

- 地址 G、L、N、O、P 不能作为自变量使用。
- 可以省略没有必要指定的地址。与省略的地址相对应的局部变量设为空值。
- 不需要按照字母顺序指定，按照字母地址格式就可以。
但是，I、J、K 必须按照字母顺序指定。
通过将参数 IJK(No.6008#7)设定为 1，即可将 I、J、K 作为第Ⅰ类自变量固定起来。在这种情况下，不需要按照字母顺序指定。